Meta-Reinforcement Learning with Zero-Shot Reinforcement Learning

Anonymous authors
Paper under double-blind review

Abstract

Meta-reinforcement learning (meta-RL) agents adapt to novel tasks from test-time experience, but require diverse training sets of environments and reward functions that are expensive to construct. Behavior Foundation Models (BFMs) learn policies from reward-free data that are capable of zero-shot RL (ZSRL): adapting to new objectives at test time when given a large reward-labeled dataset. Recent work has shown that BFM task inference can be performed online without prior data, making BFMs and meta-RL direct competitors for generalization in fixed environments. We ask whether BFMs can be extended to adapt to novel reward functions in novel environments, and identify four key limitations: environment identification, online data collection, test-time exploration-exploitation, and mixed reward supervision. We study these subproblems through toy domains, standard BFM benchmarks, and simulated humanoid locomotion, then propose a unified framework that addresses all four. The resulting method, `MetaBFM`, is a hybrid RL agent spanning meta-RL, ZSRL, and intrinsic exploration. During training, `MetaBFM` combines supervised reward-following with unsupervised reward-free learning; at test time, it interpolates between exploration, exploitation, and ZSRL-style reward inference. We evaluate `MetaBFM` on two toy meta-RL domains and at scale in MetaWorld, showing that hybrid meta-RL/ZSRL agents can learn more general behavior from the same set of reward functions and may reduce the need for future meta-RL domains to hand-design diverse training sets.

1 Introduction

Meta-Reinforcement Learning (RL) agents improve on novel tasks by adapting their decision-making from test-time experience (Beck et al., 2023). This capability relies on diverse training sets of environments and reward functions, and benefits from online data collection over many trials. Despite much algorithmic progress, meta-RL remains limited by the cost of generating these training sets. Most work focuses on toy problems where similar environments and/or reward functions are easily sampled; large-scale applications like AdA (Team et al., 2023), where a team of developers repurposed a commercial game engine to generate complex tasks and curricula, are rare exceptions. Generating a diverse set of learnable *reward functions* that covers the desired range of behavior may be inherently difficult. Reward functions are challenging to design at scale (Booth et al., 2023; Knox et al., 2023), and imperfect proxies can break down under optimization pressure (Karwowski et al., 2023). However, with hardware-accelerated simulators (Makoviychuk et al., 2021; Mittal et al., 2025), LLM-assisted coding (Karten et al., 2026), and promptable world models (Bruce et al., 2024; Che et al., 2024), the cost of generating *environments* is rapidly declining.

Behavior Foundation Models (BFMs) are an emerging class of unsupervised RL methods (Laskin et al., 2021) that train in a reward-free environment by learning to maximize many possible objectives (Agarwal et al., 2025a). At test time, a target objective is specified by model inference on a large dataset of trajectories labeled with a particular reward function. BFMs' unsupervised training and supervised testing datasets are loosely analogous to the pretraining and finetuning paradigm

of foundation models in other areas (Zhou et al., 2025). This adaptation process is often called zero-shot RL (ZSRL) (Touati et al., 2022) and has shown promising results in domains like humanoid locomotion (Tirinzoni et al.; Li et al., 2025). Meta-RL and BFMs both enable generalization without additional training, but ZSRL requires a large supervised adaptation dataset in advance, while meta-RL assumes the policy actively collects its own (far smaller) dataset. This distinction makes the two approaches applicable to different types of adaptation problems, and their test-time performance is difficult to compare directly. However, recent work by Rupf et al. (2025) demonstrated that ZSRL task inference can be done online, without a preexisting dataset, under the same conditions as meta-RL. As a result, meta-RL and BFMs can now be considered direct competitors on problems that require adaptation to novel reward functions *in a fixed environment*.

If unsupervised BFMs could behave like supervised meta-RL in the broader setting of generalizing to novel reward functions *in novel environments*, their ability to avoid meta-RL's demand for train-time reward functions could help extend generalist RL to more diverse domains. We argue that this requires BFMs to (1) identify and generalize to novel environments, (2) benefit from large-scale online data collection, (3) make exploration-exploitation tradeoffs at test time, and (4) improve when we relax the assumption that no reward functions are available during training. We study each of these subproblems in turn with detailed experiments on a challenging toy problem, standard BFM benchmarks, and large-scale simulated humanoid locomotion, arriving at a unified framework that addresses all four. The resulting method, `MetaBFM`, trains a hybrid RL agent that spans meta-RL, ZSRL, and intrinsic exploration. During training, `MetaBFM` combines supervised reward-following with unsupervised reward-free learning. At test time, the policy can interpolate between exploration, exploitation, and ZSRL-style reward inference. The main benefit is that `MetaBFM` retains meta-RL's ability to adapt to in-distribution environments and objectives, while adding the possibility of generalizing to out-of-distribution reward functions. In practice, this allows the agent to learn more from a fixed set of supervised rewards and shifts the burden from designing large training sets of reward functions to designing diverse environments.

2 Background

2.1 Meta-Reinforcement Learning

A meta-RL policy π can be evaluated by its cumulative return over the first k attempts, or episodes, in a new task. This objective creates an exploration-exploitation tradeoff at test time: the policy may sacrifice performance in early attempts to learn more about the task if exploiting that knowledge later leads to higher cumulative return (Zintgraf, 2022). Methods differ in whether this tradeoff is managed by the designer as a hyperparameter (Finn et al., 2017) or learned by the policy itself (Duan et al., 2016). Policies may also be given k free episodes to explore without consequence before being evaluated on attempt $k+1$; this **k-shot** setting creates explicit exploration and exploitation phases (Liu et al., 2021; Stadie et al., 2018).

Meta-RL's test-time adaptation is enabled by training on a diverse set of tasks. In this paper, a "task" refers specifically to a combination of an environment and an objective—or, equivalently, to a pair of parameters (c_d, c_r) that alter the transition dynamics and reward function, respectively. The basic goal of a context-based meta-learner is to use the multi-episodic trajectory[1] of interaction with the current task so far, $\tau_{:t} = \{s_0, a_0, r_1, s_1, a_1, \ldots, s_t\}$, to infer the unknown (c_d, c_r), exploit the unique characteristics of the task, and maximize returns. Methods broadly differ in whether the parameters (c_d, c_r) are estimated by supervised regression to ground-truth values (Humplik et al., 2019), as the byproduct of a self-supervised objective such as dynamics modeling (Zintgraf et al., 2021a), or implicitly through the objective of maximizing returns (Wang et al., 2016). In each case, the idea is to establish a training objective that is minimized by correctly distinguishing (c_d, c_r) and then conditioning π on the resulting representation.

[1]The main text assumes common MDP RL notation. Appendix A.1 provides a reference and a standard introduction.

Meta-RL is at its best when we can afford sample-inefficient online data collection (Team et al., 2023; Nikulin et al., 2024; Lu et al., 2023), which may be acceptable when we are willing to trade train-time compute for efficient test-time adaptation. Meta-learning is harder when restricted to a fixed dataset (Pong et al., 2022; Dorfman et al., 2020; Gao et al., 2023; Li et al., 2024), as it suffers from an extreme form of the distribution shift problem that defines offline RL more broadly (Levine et al., 2020). Methods also differ in whether task rewards are available during meta-training. Standard meta-RL is *supervised* by a distribution of reward-labeled tasks, whereas *unsupervised* meta-RL learns from reward-free interaction and only encounters rewards at test time (Beck et al., 2023). We focus on online meta-RL that spans both regimes, meta-training on a set of reward functions while simultaneously learning a reward-free BFM that supports ZSRL-style task inference at test time. This hybrid framing leads to a method distinct from prior unsupervised meta-RL (Gupta et al., 2018; Jabri et al., 2019; Pappalardo, 2026), which omits reward supervision entirely, and from prior ZSRL/BFM work, which does not meta-train across environments.

2.2 Behavior Foundation Models

BFMs are unsupervised RL agents based on successor features (Agarwal et al., 2025a; Di Ventura et al., 2025). The idea is to learn a latent-conditioned policy $\pi(a \mid s, z)$ from a reward-free dataset and infer the z that maximizes a particular task of interest from a reward-labeled dataset, without further training (Jeen et al., 2025). In general, we let $\phi : \mathcal{S} \to \mathbb{R}^d$ define latent rewards $r_z(s) = \phi(s)^\top z$ and learn successor features $\psi(s, a, z) \approx \mathbb{E}_{\pi_z}[\sum_{t \geq 0} \gamma^t \phi(s_{t+1}) \mid s_0 = s, a_0 = a]$, so that $Q_{\text{SF}}(s, a, z) = \psi(s, a, z)^\top z$ can serve as the critic in a standard off-policy actor-critic update (Lillicrap et al., 2015) over some distribution of z, e.g., $\mathcal{L}_{\text{actor}} = -\mathbb{E}_{s \sim \mathcal{D}, z \sim p(z), a \sim \pi(\cdot \mid s, z)} [Q_{\text{SF}}(s, a, z)]$. In this work, the details of training ψ and ϕ will be interchangeable and are not necessary to follow the main text, so we defer a more complete introduction to Appendix A.2. Most recent work is based on the FB algorithm (Touati & Ollivier, 2021; Touati et al., 2022), but there are many alternatives and extensions (Chua et al., 2024; Bagatella et al., 2026; Zheng et al., 2026; Jeen et al., 2024; Agarwal et al., 2025b; Park et al., 2024). Our experiments primarily use TD-JEPA (Bagatella et al., 2025) because it shares similarities with off-policy supervised RL and lets us transfer implementation details from an optimized meta-RL baseline when appropriate.

BFMs benefit from diverse training datasets with broad state-action coverage, and are most commonly applied to offline datasets originally collected by a policy that optimized for exploration (Burda et al., 2018; Laskin et al., 2021). They adapt to a test-time task by fitting $z^* = \arg\min_z \sum_{\mathcal{D}_r} (r_i - \phi(s_i)^\top z)^2$ on a reward-labeled dataset to deploy $\pi(\cdot \mid s, z^*)$, and we will refer to this inference process as zero-shot RL (ZSRL). Meta-RL's inference of c_r plays a similar role as ZSRL's inference of z^*: both parameters estimate the active reward function and therefore enable generalization to novel objectives. ZSRL relies on an existing *offline* dataset $\mathcal{D}_r$ to fit z^*, after which this parameter is fixed. In contrast, meta-RL typically infers c_r from $\tau_{:t}$ (an *online* $\mathcal{D}_r$ it has collected itself) by updating a belief between timesteps or episodes (Rakelly et al., 2019). This discrepancy makes ZSRL and meta-RL difficult to compare directly. In fact, "zero-shot RL" is, by design, unsuited to zero-shot generalization as typically defined (Section 2.1), where we expect a policy to perform well in a novel task on the first attempt with no prior data. OptiBFM (Rupf et al., 2025) helps to close this gap by demonstrating that, as in meta-RL, we can maintain a belief over z^* that is updated between timesteps or episodes.

2.3 Generalization of BFMs and Supervised Meta-RL

With OptiBFM's *online task inference* enabled, BFMs and meta-RL can be directly compared. We will use a didactic ColorGrid problem as a running example through Section 3. ColorGrid creates an $n \times n$ continuous 2D gridworld where the agent receives a reward based on the unit square cell of its current location, and the objective is to navigate to the cell with the highest reward and remain there. The underlying cell layout can be randomly permuted such that any two cells may be adjacent. In short, we can generate $(n^2)!$ distinct environments where nearly every transition is unpredictable

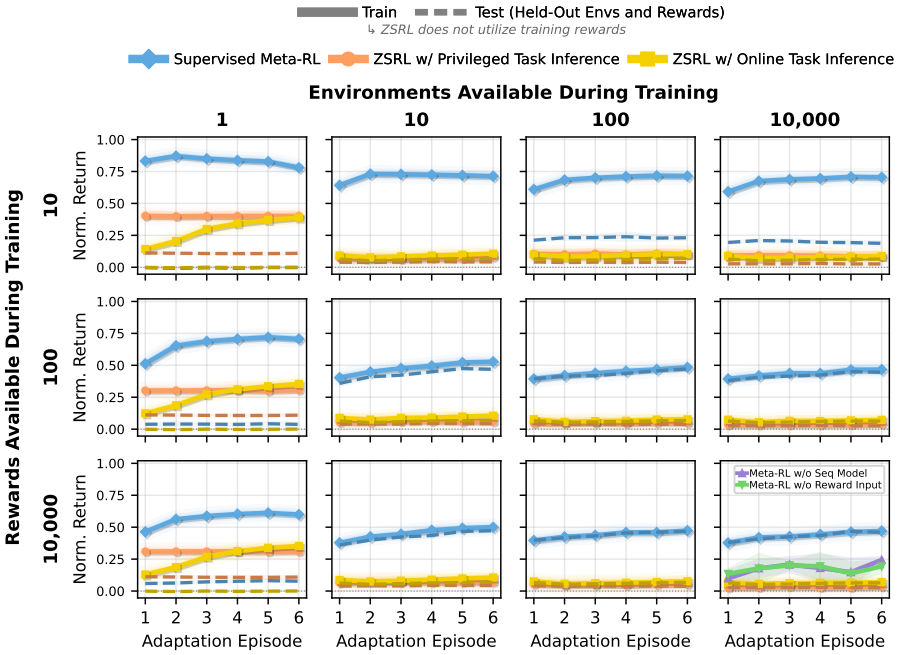

Figure 1: **BFM and Meta-RL Generalization.** In offline ColorGrid ($n = 9$), we vary training environments (columns) and rewards (rows), then evaluate $k = 6$ episode adaptation on train and held-out tasks. Meta-RL improves with training set diversity but needs many reward functions to generalize; BFMs adapt in the fixed-environment setting (left column), but fail across environments.

without a reasonable estimate of the active grid permutation, c_d. Randomly sampling a reward for each cell generates infinitely many c_r values. Additional environment details are in Appendix B.

As a first experiment, we split c_d and c_r into train/test sets and generate tasks from their cross product, measuring environment and reward generalization separately. To replicate BFMs' high-coverage offline setting, we collect large random-policy datasets. Our BFM baseline is the better of FB and TD-JEPA, while our meta-RL baseline is an off-policy variant of RL^2 (Duan et al., 2016) that makes offline training reasonable, if still slightly unfair—we return to this later. Figure 1 reports results for $n = 9$. Meta-RL follows a predictable trend: adding environments or reward functions improves generalization, but closing the train/test gap requires a diverse mixture of both. In this sense, supervised meta-RL is designed for the bottom-right corner of Fig. 1. OptiBFM allows BFMs to match the standard z^* inference procedure, though adaptation barely saturates within the $k = 6$ test episodes. In contrast, meta-RL has trained on reward functions and is incentivized to learn a narrow prior over c_r. For now, we are not concerned with the performance gap between BFMs and meta-RL where BFMs show meaningful adaptation. Instead, we focus on BFMs' relative performance: the train curves in the left column of Fig. 1 represent the established BFM setting, and performance holds as evaluated reward functions grow in a fixed environment, even though the reward labels go unused and therefore act as a test set themselves. However, BFMs immediately collapse with multiple training environments, a problem also identified by Bobrin et al. (2025).

3 BFMs as Meta-RL Agents

BFMs can adapt to unseen reward functions over k attempts without train-time rewards. However, to be useful in meta-RL, they must also generalize across environments. This is only one of several gaps: meta-RL benefits from online training, while BFMs default to offline datasets; meta-RL manages exploration and exploitation over k test episodes, while BFMs do not; and complete reward-free learning is often unnecessary in simulated online RL domains. We can usually define some reward-

labeled tasks, even if we cannot cover every behavior needed at deployment. We investigate these gaps before proposing a solution that enables a competitive hybrid between meta-RL and BFMs.

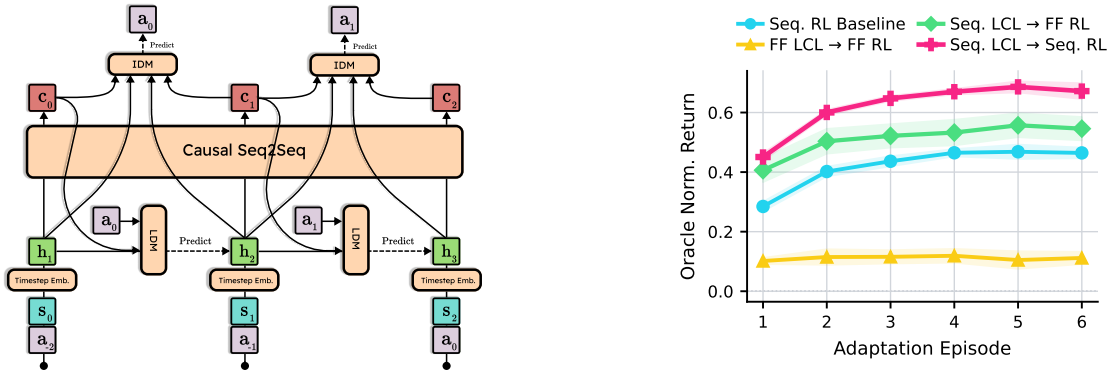

Figure 2: **Latent Context Learning. (Left)** We learn a timestep embedding h_t and sequence model $S_\theta(\tau_{:t})$ that generates an online estimate of the environment parameters, $\tilde{c}_t$. The representation is trained with self-supervised inverse and latent dynamics objectives; Algorithm 1 gives the full procedure. Action inputs a_t are necessary context to infer dynamics, but are time-shifted to avoid leaking the IDM label. **(Right)** Context features learned by a feedforward (FF) or Transformer (Seq.) S_θ are presented to either a feedforward or sequence-based supervised RL policy π in ColorGrid.

3.1 Adapting to Novel Dynamics

An RL policy that cannot condition π on estimates of the current dynamics, c_d, and reward function, c_r, learns an average policy under $p(c_d, c_r)$ that is generally not optimal for any specific task (Ghosh et al., 2019). BFMs share this problem, but z^* already plays the role of c_r, leaving us to infer c_d. Following context-based meta-RL (Beck et al., 2023), we augment each trajectory τ with a learned estimate $\tilde{c}_d$ and train $\pi(a \mid s, \tilde{c}_d)$. Because context labels are assumed to be unavailable, we use a self-supervised dynamics objective. Unlike BeliefFB (Bobrin et al., 2025), we target timestep-level online task inference with latent dynamics representations that avoid method-specific changes to the BFM update.

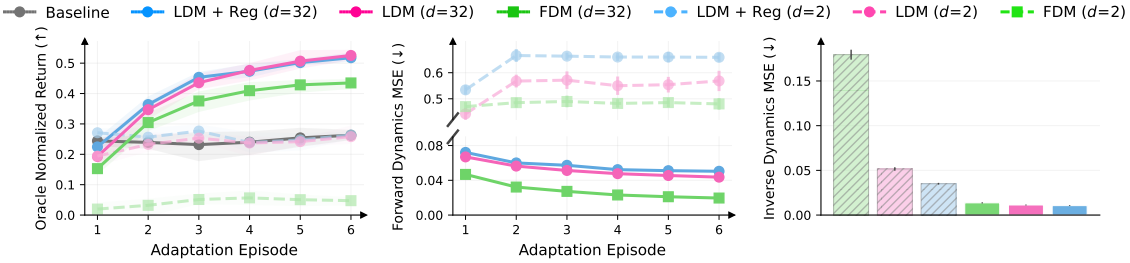

Figure 3: **LCL and Generalization.** TD-JEPA in ColorGrid $n = 5$ with various context embedding sizes and training objectives. Performance correlates with lower FDM and IDM error. LDM representations recover similar dynamics prediction as FDM without training on this objective.

Figure 2 shows our Latent Context Learning (LCL) architecture, which produces $\tilde{c}_d$ at every timestep during rollouts. We embed each timestep of the trajectory $\tau_{:t}$ to create the input sequence $\{h_0, \ldots, h_t\}$, then pass it through a causal sequence model. Let $S_\theta(\tau_{:t}) \in \mathbb{R}^d$ be the output representation at timestep t. This output is trained to identify the active dynamics through self-supervised inverse dynamics modeling (IDM) and latent dynamics modeling (LDM). Because accurate estimates of c_d improve these objectives, and because S_θ is the only representation with access to the necessary history, we treat $\tilde{c}_t = S_\theta(\tau_{:t})$ and can train a standard BFM on augmented states $\bar{s}_t = (s_t, \tilde{c}_t)$. S_θ observes action choices to infer dynamics, but we avoid leaking the inverse dynamics model label by

staggering the action input by two timesteps. We prefer latent dynamics modeling because forward dynamics modeling (FDM) scales with the size of the state space. Following latent world modeling and self-supervised sequence models (Oord et al., 2018; Schwarzer et al., 2020; 2023), we use an EMA target network for h_{t+1} labels (Grill et al., 2020), and can optionally encourage $\tilde{c}_t$ to be dissimilar across sequences in each training batch (Jajoo et al., 2026). Appendix C.1 and Algorithm 1 provide additional details. By default, we train IDM and LDM jointly; detached FDM prediction error is used only as a diagnostic metric.

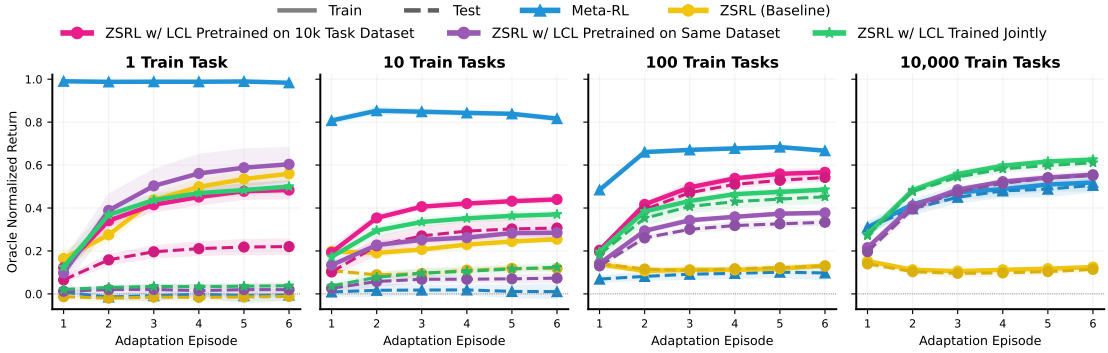

Figure 4: **Latent context learning improves BFM generalization.** We increase the number of training (c_d, c_r) task pairs with separate offline datasets. Standard TD-JEPA collapses across environments, while LCL-augmented TD-JEPA recovers adaptation and improves test performance as the training set grows. Pretraining LCL is most helpful at small training set sizes, but joint training approaches the same performance at scale.

Context estimates should improve supervised RL by reducing or eliminating the need for long-term memory policies, and Figure 2 (Right) verifies this effect in ColorGrid. By augmenting states with context estimates, we can also make BFM networks context-aware without the instability of sequence models. Figure 3 shows that, when the context representation $S_\theta(\tau) \in \mathbb{R}^{T\times d}$ has enough capacity to represent c_d, joint IDM and FDM learns accurate dynamics estimates that improve TD-JEPA performance. In Figure 4, we define train/test splits of (c_d, c_r) task pairs with separate offline datasets, and evaluate generalization as the number of training tasks grows. Since LCL and the downstream BFM agent are independent, they can be trained separately. Pretraining and freezing LCL on the largest task set improves BFM generalization at small training set sizes, but gives no clear advantage over training both systems from scratch on the same offline dataset. This is encouraging because joint training is the only realistic option when the dataset is collected online.

To further illustrate this BFM limitation and demonstrate the effectiveness of context learning, we conduct experiments on motion-tracking tasks for G1 humanoid locomotion based on BFM-Zero (Li et al., 2025). We introduce random dynamics variations through wind and gravity perturbations of varying strength. Consistent with our analysis above, BFMs take average actions across multiple dynamics settings, which can lead to performance degradation. In the in-distribution setting, we find that BFM-Multi performs even worse than BFM-Single under multi-dynamics evaluation, suggesting that BFMs suffer substantially when they cannot infer the environment dynamics. We evaluate tracking performance on the LAFAN (Harvey et al., 2020) dataset, which contains 4.6 hours of motion sequences. As shown in Table 1, context learning greatly improves performance in the in-distribution setting, suggesting its promise for cross-domain training. Surprisingly, context learning also helps to some extent in the out-of-distribution setting. We provide an illustration of BFM failures under multiple dynamics, along with additional details about the task and environment, in Appendix B.3.

3.2 Exploring, Exploiting, and Learning Online

BFM algorithms are primarily developed on offline RL benchmarks (Laskin et al., 2021) with behavior datasets generated by an online policy optimizing an intrinsic reward (Burda et al., 2018; Seo et al., 2021; Raileanu & Rocktäschel, 2020). BFMs rely on high state-action coverage and struggle to learn

Table 1: **BFMs with latent context learning improve humanoid motion tracking.** We compare motion-tracking metrics for humanoid locomotion under in-distribution and out-of-distribution dynamics. BFM-Single trains without wind/gravity randomization and can be viewed as an in-distribution oracle for BFM-Multi, or as a worst-case baseline under OOD dynamics.

Method	In-Distribution				Out-of-Distribution			
	mpjpe-l(↓)	vel. dist.(↓)	accel. dist.(↓)	emd(↓)	mpjpe-l(↓)	vel. dist.(↓)	accel. dist.(↓)	emd(↓)
BFM-Single	117.2	1.64	2.75	1.01	194.7	2.81	4.83	1.80
BFM-Multi	132.7	1.77	2.96	1.13	151.6	2.13	3.63	1.39
FDM+BFM-Multi	125.3	1.73	2.90	1.09	149.3	2.09	3.54	1.37
LDM+BFM-Multi	**121.5**	**1.68**	**2.85**	**1.05**	**145.9**	**2.06**	**3.52**	**1.36**

online because the actor objective $\mathcal{L}_{\text{actor}}$ collapses onto narrow behavior. Appendix E.1 explores the difficulty of online learning and task inference in a standard locomotion benchmark. DVFB (Sun et al., 2025) addresses dataset diversity by maximizing a combination of Q_{SF} and a supervised critic trained on intrinsic curiosity rewards, which we will call Q_R (Equation 1). Recent applications of BFMs to humanoid robotics (Tirinzoni et al.; Li et al., 2025) use a similar idea, but derive Q_R rewards from behavior in an expert motion-capture dataset.

$$\mathcal{L}_{\text{DVFB}} = -\mathbb{E}_{s\sim\mathcal{D},\, z\sim p(z),\, a\sim\pi(\cdot|s,z)}\big[\, Q_{\text{SF}}(s,a,z) \;+\; \alpha\, Q_R(s,a,z)\,\big]. \tag{1}$$

In meta-RL problems, we can take this idea further. First, we replace the *global* intrinsic reward—one that decreases as a function of total training steps, such as RND (Burda et al., 2018)—with an *episodic* reward that decays within a k-episode meta-trial (Henaff et al., 2023). An agent maximizing this reward is incentivized to explore new states across each episode at test time. We use an intrinsic reward that combines an off-the-shelf episodic bonus (NGU (Badia et al., 2020) or E3B (Henaff et al., 2022)) on LCL timestep features h_t (Fig. 2) with the LCL prediction error from the previous timestep. The latter is a core meta-RL idea (Zintgraf et al., 2021b; Zhang et al., 2021): it incentivizes exploration that improves the $\tilde{c}_d$ estimate, which can also improve in-context. Second, because episodic rewards are non-Markov, Q_R and π should have memory and take LCL-augmented trajectories $\bar{\tau}$ as input. Finally, we sample α during training and condition Q_R on the active value:

$$\mathcal{L}_{\alpha\text{-BFM}} = -\mathbb{E}_{\substack{\tau\sim\mathcal{D},\, z\sim p(z),\\ \alpha\sim p(\alpha),\, a\sim\pi(\cdot|\bar{\tau},z,\alpha)}}\Big[\,(1-\alpha)\, Q_{\text{SF}}(\bar{s},a,z) \;+\; \alpha\, Q_R(\bar{\tau},a,z,\alpha)\,\Big]. \tag{2}$$

The result is a policy that generates more diverse data during training while allowing α to vary at test time. Due to our choice of intrinsic reward, $\alpha = 1$ drives the policy to explore new behavior each episode in hopes of (1) finding rewards that improve OptiBFM's online task inference of z^* and (2) improving the context estimate $\tilde{c}_d$. In contrast, $\alpha = 0$ exploits that knowledge to maximize returns under z^*. We can schedule α to recover meta-RL's k-shot exploration-exploitation tradeoff at test time, as demonstrated in Figure 5.

3.3 Learning from Available Rewards

BFMs assume access to zero reward labels during training. While this may be necessary when learning offline, such as from a web dataset that was not intended for RL, the constraint is unnecessary when learning online in simulated environments. When we create meta-RL or multi-task RL training domains, we can usually create some reward-labeled tasks; the challenge is creating enough to cover all behaviors we may want during deployment. Adding training tasks should improve generalization with diminishing returns, creating a clear path to improving the agent by improving its dataset. Ideally, we would bring this relationship to BFMs. Each new train-time objective can help span the space of possible behaviors and supervise learning progress, while unsupervised learning fills in gaps and finds solutions to objectives we had not considered.

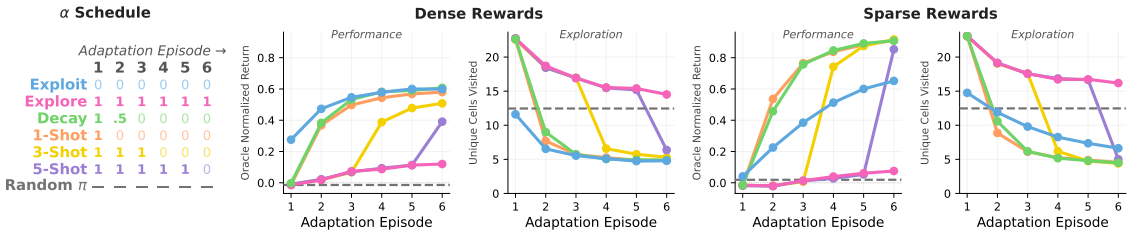

Figure 5: **Exploring at Test Time.** ColorGrid ($n = 5$) policies trained by Eq. 2 can vary α over $k = 6$ test episodes. High α explores more of the state space, improving online task estimates, which can then be exploited to outperform the standard BFM policy ($\alpha = 0$, blue) when rewards are sparse.

We can approach this problem from two directions. The first is to let reward labels improve the BFM update itself. For example, BFMs learn features $\phi(s)$ and assume $\phi(s)^\top z = r$; when reward labels are available during training, we can enforce this relationship directly with an auxiliary loss. It has also become standard to sample training $z \sim p(z)$ from recent values of $\phi(s)$, implicitly assuming evaluation on sparse state-based goal-reaching (Touati & Ollivier, 2021; Touati et al., 2022). With reward labels, we can extend this idea to arbitrary reward functions by using online task inference along the labeled trajectory to estimate z^*, as if the same states and rewards appeared at test time. Appendix E.2 evaluates both ideas in ColorGrid and DMC ExoRL, finding that they make ϕ more predictive of rewards but have mixed effects on policy performance. A second approach is to restore supervised meta-RL where rewards are available and create a true hybrid method. We modify the "intrinsic" critic Q_R from Section 3.2 to learn from a weighted combination of intrinsic rewards, r_i, and extrinsic (supervised) rewards r_e:

$$Q_R(\bar{\tau}, a, z, \alpha, \beta) \approx \underbrace{(1-\beta)\, r_i + \beta\, r_e}_{\text{Intrinsic and Extrinsic Reward}} + \gamma\, Q_R\big(\bar{\tau}', a', z, \alpha, \beta\big) \tag{3}$$

Where $\bar{\tau}'$ and a' are the next timestep in the standard TD backup. We are then free to repeat the idea from Equation 2 of sampling the weight β during training and conditioning the actor and critic on the current value so that it may also be adjusted at deployment:

$$\mathcal{L}_{\alpha\beta\text{-BFM}} = -\,\mathbb{E}_{\substack{\tau\sim\mathcal{D},\, z\sim p(z),\\ \alpha\sim p(\alpha),\, \beta\sim p(\beta),\\ a\sim\pi(\cdot|\bar{\tau}, z, \alpha, \beta)}} \Big[(1-\alpha)\, Q_{\text{SF}}(\bar{s}, a, z) \;+\; \alpha\, Q_R(\bar{\tau}, a, z, \alpha, \beta) \Big] \tag{4}$$

When $\beta = 0$, we recover the exploratory ZSRL objective of Eq. 2, and when $\alpha = \beta = 0$, we recover standard ZSRL. Most importantly, when $\alpha = \beta = 1$, we recover supervised meta-RL. In practice, this makes it possible to compete with a standard supervised meta-RL baseline on in-distribution tasks, while opening the possibility of generalizing via z to novel tasks when meta-RL fails. Figure 6 visualizes this space of possible test-time settings.

4 MetaBFM

We can now combine the pieces from Section 3 into a unified update, `MetaBFM`, that captures the in-distribution performance of supervised meta-RL and the out-of-distribution generalization of unsupervised BFMs. At each training step, we first update the Latent Context Learning module with its self-contained objective (Algorithm 1). LCL's primary role is to augment each state with a context estimate $\tilde{c}_d$, allowing the BFM networks to learn from data generated by multiple environment dynamics (Section 3.1). LCL also produces per-timestep representations that can be passed to off-the-shelf episodic curiosity methods to compute intrinsic rewards r_i for the trajectory. We then update the BFM networks, excluding the actor π, over a distribution $p(z)$. Any actions from π

required for the BFM update are sampled from $\pi(\cdot \mid \bar{\tau}, z, \alpha = 0, \beta = 0)$, which recovers the original BFM policy; the remaining details are unchanged, and many ZSRL algorithms are interchangeable here, though we use TD-JEPA. Next, we update the critic head Q_R with a standard off-policy backup on weighted intrinsic and extrinsic rewards (Equation 3) and compute the actor loss $\mathcal{L}_{\alpha\beta\text{-BFM}}$ (Equation 4), both over samples from $p(\alpha, \beta)$. We collect trajectories from the online environment by sampling from the same distribution of z, α, and β.

While this update is more expensive than training the BFM component in isolation, in practice we share network backbones for efficiency. For example, π and Q_R share a Transformer backbone to enable long-term memory. Many such training details are copied directly from a strong off-policy meta-RL implementation (Grigsby et al., 2024), and Appendix C provides additional discussion. To make extrinsic rewards optional, we force $\beta = 0$ when collecting or training on reward-free trajectories. Early layers that take extrinsic reward inputs to Q_R and π have their outputs multiplied by β, allowing us to insert and automatically ignore an arbitrary placeholder reward.

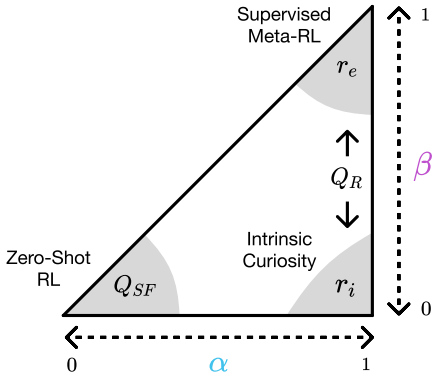

Figure 6: **A Unified Objective.** The $\mathcal{L}_{\alpha\beta\text{-BFM}}$ loss (Eq. 4) interpolates between zero-shot RL, intrinsic curiosity, and supervised meta-RL. α and β can be configured to adjust generalization behavior.

Figure 1 showed that BFMs can learn adaptive behavior from limited training domains where supervised meta-RL overfits, but also that meta-RL improves with diverse datasets while BFMs do not. `MetaBFM`'s main goal is to allow for test-time configuration to take advantage of both of these methods strengths while being able to avoid their weaknesses. Figure 7 Left returns to this experiment and shows that `MetaBFM` captures both modes: a single policy can be deployed in "ZSRL mode" ($\alpha = 0, \beta = 0$) when meta-RL fails to generalize, or in "reward-following mode" ($\alpha = 1, \beta = 1$) when the dataset is diverse enough to support it.

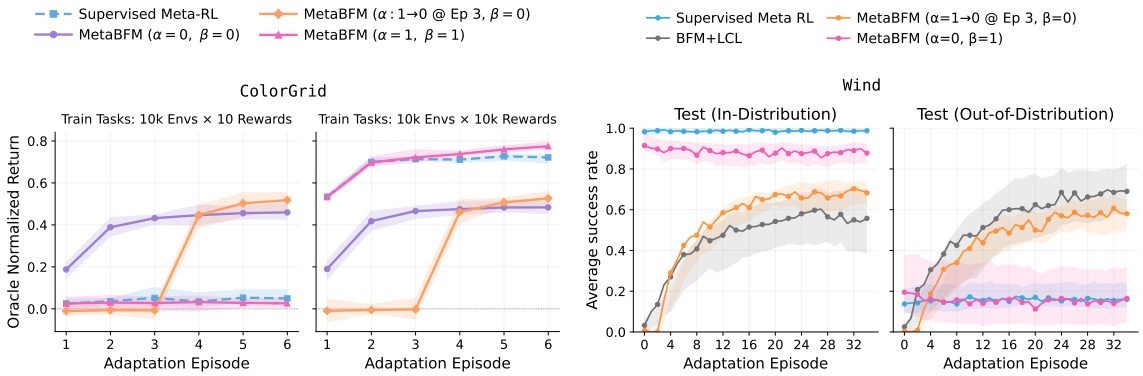

Figure 7: **Configurable MetaBFM Inference.** (**Left**) When learning from limited training objectives, `MetaBFM` in ZSRL mode ($\beta = 0$) avoids supervised meta-RL's overfitting. (**Center Left**) With more objectives, supervised mode recovers the meta-RL baseline. (**Center Right**) On held-out in-distribution Wind tasks, `MetaBFM` remains competitive with supervised meta-RL. (**Right**) On out-of-distribution Wind tasks, ZSRL-style inference adapts over long adaptation horizons.

An extension of learning more general behavior from fewer train-time objectives is generalizing to test-time objectives that are clearly out of distribution from the supervised signals seen during training. We study this with a variant of the Wind environment (Dorfman et al., 2020; Ni et al., 2022), a common benchmark for OOD generalization. Agents navigate from the origin toward a randomly placed goal, c_r, while adjusting for wind in a random direction, c_d. The training set contains all wind directions but excludes goals in the upper-right quadrant. At test time, we therefore

evaluate both held-out in-distribution tasks (any wind direction, goals in the three training quadrants) and out-of-distribution tasks (any wind direction, goals in the excluded quadrant). Figure 7 Right shows the adaptation curves for each set. `MetaBFM` significantly outperforms the supervised baseline on the OOD test set when configured to maximize successor-feature values, while nearly matching its performance on in-distribution tasks where train-time rewards are useful. Perhaps the most interesting OOD result is the gap between the train-time trajectory length of $k = 2$ episodes and the length at which online task inference saturates ($k > 20$). This kind of length extrapolation is rare in black-box meta-RL and may be enabled by offloading the usual context estimation of c_r outside of the policy weights and into ZSRL's z^*.

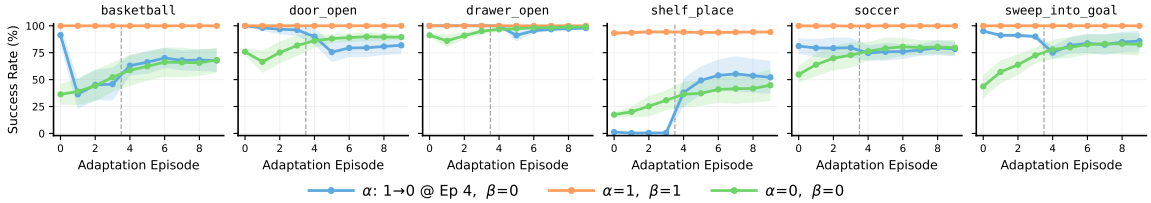

Figure 8: **Online Learning in MetaWorld ML1.** `MetaBFM` evaluated in the three main inference configurations on six MetaWorld ML1 problems.

Finally, we evaluate `MetaBFM` in the more complex continuous-control benchmark of MetaWorld (Yu et al., 2020), using the hardware-accelerated variant from MTBench (Joshi et al., 2025). Despite learning to maximize a space of actor objectives at train and test time, `MetaBFM` converges to near-perfect performance on six ML1 tasks, which measure generalization over hidden-parameter variations of a single tabletop manipulation skill. Episodic curiosity ($\alpha = 1, \beta = 0$) is also strongly correlated with success rate on many ML1 tasks. Figure 9 evaluates the full multi-task ML45 setting. Due to computational constraints, we seed these runs with an initial replay buffer of demonstrations generated by randomly selected checkpoints from a multi-day meta-RL training run with exploration noise enabled. We report performance after 1M gradient steps for baselines trained offline on this dataset, as well as meta-RL and `MetaBFM` agents that continue collecting online experience for exploration and coverage. `MetaBFM` again recovers supervised meta-RL as an approximate lower bound on performance. However, it remains unable to generalize, in either reward-following or ZSRL inference modes, to MetaWorld's notoriously disjoint test set of five held-out manipulation tasks. Because `MetaBFM` can learn from reward-free trajectories, expanding ML45 with additional environment diversity may be a path toward closing this gap.

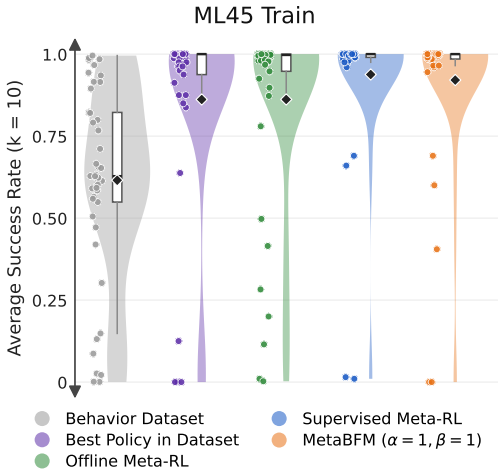

Figure 9: **MetaWorld ML45.** Performance across 45 train tasks when replay buffers are initialized from a mixture of policies.

5 Conclusion

Meta-RL's reliance on diverse reward functions has bottlenecked its scaling, even as environments become cheaper to generate. Behavior Foundation Models sidestep this by decoupling environment and reward representations, but standard ZSRL has kept BFMs outside meta-RL's online adaptation regime. Building on OptiBFM (Rupf et al., 2025), we extend the unsupervised BFM paradigm to generalize across both novel rewards and novel environments. `MetaBFM` achieves this through a self-predictive latent context for environment identification, intrinsic curiosity that drives both online data collection and test-time exploration, and mixed supervised-unsupervised reward training that recovers in-distribution meta-RL performance while enabling generalization beyond the training set.

References

Siddhant Agarwal, Caleb Chuck, Harshit Sikchi, Jiaheng Hu, Max Rudolph, Scott Niekum, Peter Stone, and Amy Zhang. A unified framework for unsupervised reinforcement learning algorithms. In *Workshop on Reinforcement Learning Beyond Rewards @ Reinforcement Learning Conference 2025*, 2025a. URL https://openreview.net/forum?id=tQtO75p5HA.

Siddhant Agarwal, Harshit Sikchi, Peter Stone, and Amy Zhang. Proto successor measure: Representing the behavior space of an rl agent. In *International Conference on Machine Learning*, pp. 566–586. PMLR, 2025b.

Adrià Puigdomènech Badia, Pablo Sprechmann, Alex Vitvitskyi, Daniel Guo, Bilal Piot, Steven Kapturowski, Olivier Tieleman, Martín Arjovsky, Alexander Pritzel, Andew Bolt, et al. Never give up: Learning directed exploration strategies. *arXiv preprint arXiv:2002.06038*, 2020.

Marco Bagatella, Matteo Pirotta, Ahmed Touati, Alessandro Lazaric, and Andrea Tirinzoni. Td-jepa: Latent-predictive representations for zero-shot reinforcement learning. *arXiv preprint arXiv:2510.00739*, 2025.

Marco Bagatella, Thomas Rupf, Georg Martius, and Andreas Krause. Soft forward-backward representations for zero-shot reinforcement learning with general utilities. *arXiv preprint arXiv:2602.06769*, 2026.

Jacob Beck, Risto Vuorio, Evan Zheran Liu, Zheng Xiong, Luisa Zintgraf, Chelsea Finn, and Shimon Whiteson. A survey of meta-reinforcement learning. *arXiv preprint arXiv:2301.08028*, 2023.

Maksim Bobrin, Ilya Zisman, Alexander Nikulin, Vladislav Kurenkov, and Dmitry Dylov. Zero-shot adaptation of behavioral foundation models to unseen dynamics. *arXiv preprint arXiv:2505.13150*, 2025.

Serena Booth, W Bradley Knox, Julie Shah, Scott Niekum, Peter Stone, and Alessandro Allievi. The perils of trial-and-error reward design: misdesign through overfitting and invalid task specifications. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 5920–5929, 2023.

Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.

Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. *arXiv preprint arXiv:1810.12894*, 2018.

Haoxuan Che, Xuanhua He, Quande Liu, Cheng Jin, and Hao Chen. Gamegen-x: Interactive open-world game video generation. *arXiv preprint arXiv:2411.00769*, 2024.

Raymond Chua, Arna Ghosh, Christos Kaplanis, Blake A Richards, and Doina Precup. Learning successor features the simple way. *Advances in Neural Information Processing Systems*, 37: 49957–50030, 2024.

Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv preprint arXiv:2205.14135*, 2022.

Hao Liu Michael Laskin Pieter Abbeel Alessandro Lazaric Lerrel Pinto Denis Yarats, David Brandfonbrener. Don't change the algorithm, change the data: Exploratory data for offline reinforcement learning. *arXiv preprint arXiv:2201.13425*, 2022.

Jacopo Di Ventura, Jan Felix Kleuker, Aske Plaat, and Thomas Moerland. A unified framework for zero-shot reinforcement learning. *arXiv preprint arXiv:2510.20542*, 2025.

Ron Dorfman, Idan Shenfeld, and Aviv Tamar. Offline meta learning of exploration. *arXiv preprint arXiv:2008.02598*, 2020.

Yan Duan, John Schulman, Xi Chen, Peter L Bartlett, Ilya Sutskever, and Pieter Abbeel. Rl ^{2}: Fast reinforcement learning via slow reinforcement learning. *arXiv preprint arXiv:1611.02779*, 2016.

Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pp. 1126–1135. PMLR, 2017.

Yunkai Gao, Rui Zhang, Jiaming Guo, Fan Wu, Qi Yi, Shaohui Peng, Siming Lan, Ruizhi Chen, Zidong Du, Xing Hu, et al. Context shift reduction for offline meta-reinforcement learning. *Advances in Neural Information Processing Systems*, 36:80024–80043, 2023.

Dibya Ghosh, Abhishek Gupta, Ashwin Reddy, Justin Fu, Coline Devin, Benjamin Eysenbach, and Sergey Levine. Learning to reach goals via iterated supervised learning. *arXiv preprint arXiv:1912.06088*, 2019.

Jake Grigsby, Justin Sasek, Samyak Parajuli, Daniel Adebi, Amy Zhang, and Yuke Zhu. Amago-2: Breaking the multi-task barrier in meta-reinforcement learning with transformers. *Advances in Neural Information Processing Systems*, 37:87473–87508, 2024.

Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al. Bootstrap your own latent-a new approach to self-supervised learning. *Advances in neural information processing systems*, 33:21271–21284, 2020.

Abhishek Gupta, Benjamin Eysenbach, Chelsea Finn, and Sergey Levine. Unsupervised meta-learning for reinforcement learning. *arXiv preprint arXiv:1806.04640*, 2018.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

Félix G. Harvey, Mike Yurick, Derek Nowrouzezahrai, and Christopher Pal. Robust motion in-betweening. 39(4), 2020.

Mikael Henaff, Roberta Raileanu, Minqi Jiang, and Tim Rocktäschel. Exploration via elliptical episodic bonuses. *Advances in Neural Information Processing Systems*, 35:37631–37646, 2022.

Mikael Henaff, Minqi Jiang, and Roberta Raileanu. A study of global and episodic bonuses for exploration in contextual mdps. In *International Conference on Machine Learning*, pp. 12972–12999. PMLR, 2023.

Jan Humplik, Alexandre Galashov, Leonard Hasenclever, Pedro A Ortega, Yee Whye Teh, and Nicolas Heess. Meta reinforcement learning as task inference. *arXiv preprint arXiv:1905.06424*, 2019.

Allan Jabri, Kyle Hsu, Abhishek Gupta, Ben Eysenbach, Sergey Levine, and Chelsea Finn. Unsupervised curricula for visual meta-reinforcement learning. *Advances in Neural Information Processing Systems*, 32, 2019.

Pranaya Jajoo, Harshit Sikchi, Siddhant Agarwal, Amy Zhang, Scott Niekum, and Martha White. Regularized latent dynamics prediction is a strong baseline for behavioral foundation models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=jdL6WB5jHZ.

Scott Jeen, Tom Bewley, and Jonathan M Cullen. Zero-shot reinforcement learning from low quality data. *Advances in Neural Information Processing Systems*, 37:16894–16942, 2024.

Scott Jeen, Tom Bewley, and Jonathan M Cullen. Zero-shot reinforcement learning under partial observability. *arXiv preprint arXiv:2506.15446*, 2025.

Viraj Joshi, Zifan Xu, Bo Liu, Peter Stone, and Amy Zhang. Benchmarking massively parallelized multi-task reinforcement learning for robotics tasks. *arXiv preprint arXiv:2507.23172*, 2025.

Seth Karten, Rahul Dev Appapogu, and Chi Jin. Automatic generation of high-performance rl environments. *arXiv preprint arXiv:2603.12145*, 2026.

Jacek Karwowski, Oliver Hayman, Xingjian Bai, Klaus Kiendlhofer, Charlie Griffin, and Joar Max Viktor Skalse. Goodhart's law in reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2023.

W Bradley Knox, Alessandro Allievi, Holger Banzhaf, Felix Schmitt, and Peter Stone. Reward (mis) design for autonomous driving. *Artificial Intelligence*, 316:103829, 2023.

Michael Laskin, Denis Yarats, Hao Liu, Kimin Lee, Albert Zhan, Kevin Lu, Catherine Cang, Lerrel Pinto, and Pieter Abbeel. URLB: Unsupervised reinforcement learning benchmark. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL https://openreview.net/forum?id=lwrPkQP_is.

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

Lanqing Li, Hai Zhang, Xinyu Zhang, Shatong Zhu, Yang Yu, Junqiao Zhao, and Pheng-Ann Heng. Towards an information theoretic framework of context-based offline meta-reinforcement learning. *Advances in Neural Information Processing Systems*, 37:75642–75667, 2024.

Yitang Li, Zhengyi Luo, Tonghe Zhang, Cunxi Dai, Anssi Kanervisto, Andrea Tirinzoni, Haoyang Weng, Kris Kitani, Mateusz Guzek, Ahmed Touati, et al. Bfm-zero: A promptable behavioral foundation model for humanoid control using unsupervised reinforcement learning. *arXiv preprint arXiv:2511.04131*, 2025.

Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

Evan Z Liu, Aditi Raghunathan, Percy Liang, and Chelsea Finn. Decoupling exploration and exploitation for meta-reinforcement learning without sacrifices. In *International conference on machine learning*, pp. 6925–6935. PMLR, 2021.

Chris Lu, Yannick Schroecker, Albert Gu, Emilio Parisotto, Jakob Foerster, Satinder Singh, and Feryal Behbahani. Structured state space models for in-context reinforcement learning. *arXiv preprint arXiv:2303.03982*, 2023.

Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, et al. Isaac gym: High performance gpu-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021.

Reginald McLean, Evangelos Chatzaroulas, Luc McCutcheon, Frank Röder, Tianhe Yu, Zhanpeng He, KR Zentner, Ryan Julian, Jordan K Terry, Isaac Woungang, et al. Meta-world+: An improved, standardized, rl benchmark. *arXiv preprint arXiv:2505.11289*, 2025.

Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, Antonio Serrano-Munoz, Xinjie Yao, René Zurbrügg, Nikita Rudin, et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.

Tianwei Ni, Benjamin Eysenbach, and Ruslan Salakhutdinov. Recurrent model-free rl can be a strong baseline for many pomdps, 2022.

Alexander Nikulin, Vladislav Kurenkov, Ilya Zisman, Artem Agarkov, Viacheslav Sinii, and Sergey Kolesnikov. Xland-minigrid: Scalable meta-reinforcement learning environments in jax. *Advances in Neural Information Processing Systems*, 37:43809–43835, 2024.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

Octavio Pappalardo. Unsupervised learning of efficient exploration: Pre-training adaptive policies via self-imposed goals. *arXiv preprint arXiv:2601.19810*, 2026.

Seohong Park, Tobias Kreiman, and Sergey Levine. Foundation policies with hilbert representations. *arXiv preprint arXiv:2402.15567*, 2024.

Vitchyr H Pong, Ashvin V Nair, Laura M Smith, Catherine Huang, and Sergey Levine. Offline meta-reinforcement learning with online self-supervision. In *International Conference on Machine Learning*, pp. 17811–17829. Pmlr, 2022.

Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

Roberta Raileanu and Tim Rocktäschel. Ride: Rewarding impact-driven exploration for procedurally-generated environments. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=rkg-TJBFPB.

Kate Rakelly, Aurick Zhou, Chelsea Finn, Sergey Levine, and Deirdre Quillen. Efficient off-policy meta-reinforcement learning via probabilistic context variables. In *International conference on machine learning*, pp. 5331–5340. PMLR, 2019.

Thomas Rupf, Marco Bagatella, Marin Vlastelica, and Andreas Krause. Optimistic task inference for behavior foundation models. *arXiv preprint arXiv:2510.20264*, 2025.

Max Schwarzer, Ankesh Anand, Rishab Goel, R Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. *arXiv preprint arXiv:2007.05929*, 2020.

Max Schwarzer, Johan Samir Obando Ceron, Aaron Courville, Marc G Bellemare, Rishabh Agarwal, and Pablo Samuel Castro. Bigger, better, faster: Human-level atari with human-level efficiency. In *International Conference on Machine Learning*, pp. 30365–30380. PMLR, 2023.

Younggyo Seo, Lili Chen, Jinwoo Shin, Honglak Lee, Pieter Abbeel, and Kimin Lee. State entropy maximization with random encoders for efficient exploration. In *International conference on machine learning*, pp. 9443–9454. PMLR, 2021.

Bradly C Stadie, Ge Yang, Rein Houthooft, Xi Chen, Yan Duan, Yuhuai Wu, Pieter Abbeel, and Ilya Sutskever. Some considerations on learning to explore via meta-reinforcement learning. *arXiv preprint arXiv:1803.01118*, 2018.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Jingbo Sun, Songjun Tu, qichao Zhang, Haoran Li, Xin Liu, Yaran Chen, Ke Chen, and Dongbin Zhao. Unsupervised zero-shot reinforcement learning via dual-value forward-backward representation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=0QnKnt411O.

Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

Adaptive Agent Team, Jakob Bauer, Kate Baumli, Satinder Baveja, Feryal Behbahani, Avishkar Bhoopchand, Nathalie Bradley-Schmieg, Michael Chang, Natalie Clay, Adrian Collister, et al. Human-timescale adaptation in an open-ended task space. *arXiv preprint arXiv:2301.07608*, 2023.

Andrea Tirinzoni, Ahmed Touati, Jesse Farebrother, Mateusz Guzek, Anssi Kanervisto, Yingchen Xu, Alessandro Lazaric, and Matteo Pirotta. Zero-shot whole-body humanoid control via behavioral foundation models. In *The Thirteenth International Conference on Learning Representations*.

Ahmed Touati and Yann Ollivier. Learning one representation to optimize all rewards. *Advances in Neural Information Processing Systems*, 34:13–23, 2021.

Ahmed Touati, Jérémy Rapin, and Yann Ollivier. Does zero-shot reinforcement learning exist? *arXiv preprint arXiv:2209.14935*, 2022.

Jane X Wang, Zeb Kurth-Nelson, Dhruva Tirumala, Hubert Soyer, Joel Z Leibo, Remi Munos, Charles Blundell, Dharshan Kumaran, and Matt Botvinick. Learning to reinforcement learn. *arXiv preprint arXiv:1611.05763*, 2016.

Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on robot learning*, pp. 1094–1100. PMLR, 2020.

Jin Zhang, Jianhao Wang, Hao Hu, Tong Chen, Yingfeng Chen, Changjie Fan, and Chongjie Zhang. Metacure: Meta reinforcement learning with empowerment-driven exploration. In *International Conference on Machine Learning*, pp. 12600–12610. PMLR, 2021.

Chongyi Zheng, Royina Karegoudra Jayanth, and Benjamin Eysenbach. Can we really learn one representation to optimize all rewards? *arXiv preprint arXiv:2602.11399*, 2026.

Ce Zhou, Qian Li, Chen Li, Jun Yu, Yixin Liu, Guangjing Wang, Kai Zhang, Cheng Ji, Qiben Yan, Lifang He, et al. A comprehensive survey on pretrained foundation models: A history from bert to chatgpt. *International Journal of Machine Learning and Cybernetics*, 16(12):9851–9915, 2025.

Luisa Zintgraf. *Fast adaptation via meta reinforcement learning*. PhD thesis, University of Oxford, 2022.

Luisa Zintgraf, Sebastian Schulze, Cong Lu, Leo Feng, Maximilian Igl, Kyriacos Shiarlis, Yarin Gal, Katja Hofmann, and Shimon Whiteson. Varibad: Variational bayes-adaptive deep rl via meta-learning. *Journal of Machine Learning Research*, 22(289):1–39, 2021a.

Luisa M Zintgraf, Leo Feng, Cong Lu, Maximilian Igl, Kristian Hartikainen, Katja Hofmann, and Shimon Whiteson. Exploration in approximate hyper-state space for meta reinforcement learning. In *International Conference on Machine Learning*, pp. 12991–13001. PMLR, 2021b.

Appendix

A Additional Background

A.1 Reinforcement Learning

A Markov Decision Process (MDP) (Puterman, 2014) is defined as a tuple $(\mathcal{S}, \mathcal{A}, \mu, P, r)$ where $\mathcal{S}$ is called the state space, $\mathcal{A}$ is the action space, μ is the initial state distribution, $P : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ is the transition function that maps a given state and action to a distribution of next states and $r : \mathcal{S} \to \mathbb{R}$ is the scalar reward function. The agent executes a policy, $\pi : \mathcal{S} \to \Delta(\mathcal{A})$, to interact with the environment. The Reinforcement Learning (RL) objective is to find the optimal policy, π^*, defined as,

$$\pi^* = \text{argmax}_\pi \mathbb{E}_\pi \left[\sum_t \gamma^t r(s_t) \right] \tag{5}$$

A.2 Behavior Foundation Models

A Behavior Foundation Model(BFM), for a given dynamical system, allows an agent to produce near-optimal policies for any given reward function using little or no additional computation. As a result, this paradigm has often been referred to as *Zero-Shot Reinforcement Learning*. Specifically, in this work, we consider successor-measure-based BFMs that learn to represent successor measures for a wide range of policies. Mathematically, the successor measure for a policy is defined as,

$$M^{\pi}(s,a,X) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}\mathbb{P}^{\pi}(s_t \in X | s_0 = s, a_0 = a)\right], \forall X \subset \mathcal{S}. \tag{6}$$

BFMs represent M^{π_z} (successor measure for policy parameterized using z) as $M^{\pi_z}(s,a,s^+) = \psi(s,a,z)^T\phi(s^+)$. Connections have been drawn (Touati & Ollivier, 2021; Agarwal et al., 2025b;a) between this representation and successor features with ψ being the successor feature for the state feature $\mathbb{E}_{\rho}[\phi\phi^T]\phi(s)$. Methods like (Touati & Ollivier, 2021; Agarwal et al., 2025b; Jajoo et al., 2026) use the following objective to learn these representations,

$$\begin{aligned}\mathcal{L}(\phi,\psi) = &-\mathbb{E}_{s,a,s'\sim\rho}[\psi(s,a,z)^T\phi(s')] \\ &+ \frac{1}{2}\mathbb{E}_{s,a,s'\sim\rho,s^+\sim\rho}[(\psi(s,a,z)^T\phi(s^+) - \gamma\bar{\psi}(s',\pi_z(s'),z)^T\bar{\phi}(s^+))^2]\end{aligned} \tag{7}$$

where ρ is the offline dataset. These BFMs consider tasks defined by rewards, $r_z(s) = \phi(s)^T z$ for which the corresponding optimal Q function can be written as, $Q_z^*(s,a) = \psi(s,a,z)^T z$ with the optimal policy $\pi_z = \text{argmax}_a Q_z^*(s,a) = \text{argmax}_a \psi(s,a,z)^T z$. Overall, BFMs can be represented using (ϕ,ψ,π_z). All the three networks ϕ, ψ and π_z are learnt using reward-free interactions with ϕ and ψ being trained using Equation 7 and π_z as the optimal policy for the reward $r_z(s) = \phi(s)^T z$.

At inference, when a reward function is provided, the BFMs simply need to find the z for the given reward function as the corresponding policy π_z is already trained to be optimal for r_z. This z can be inferred simply by linear regression on reward samples, i.e. $z^* = \text{argmin}_z \mathbb{E}_{\rho}[(r(s) - \phi(s)^T z)^2]$. Several methods (Touati & Ollivier, 2021; Agarwal et al., 2025b; Jajoo et al., 2026; Park et al., 2024; Bagatella et al., 2025) have been proposed with a similar construction of BFMs.

B Environment Details

B.1 Toy Meta-RL

B.1.1 ColorGrid

ColorGrid is an $n \times n$ continuous-control gridworld in which the agent acts on grid of colors, with a hidden reward grid c_r defining a scalar payoff at each cell. Rather than observing this underlying color grid, the agent's observations are produced from a permuted color grid produced by a hidden cell permutation c_d. Each task is therefore a pair (c_d, c_r), a construction that disentangles two independent sources of partial observability the agent must reason about within a trial.

Dynamics and reward. The agent's true state is a continuous position $s_t \in [0,n]^2$, advanced by a 2D velocity action $a_t \in [-1,1]^2$ with $s_{t+1} = \text{clip}(s_t + a_t, 0, n)$, and reward is read from the true grid as $r_t = R[\lfloor s_t^{(y)} \rfloor, \lfloor s_t^{(x)} \rfloor]$. Actions therefore act faithfully on the true grid, and only the position readout to the agent is scrambled by c_d. The integer part of the agent's position is relabelled according to c_d, while the sub-cell fraction passes through unchanged. Smooth motion within a true cell therefore looks smooth in observation space, but the moment the

agent crosses a true-cell boundary the cursor teleports to an unrelated observation cell (Figure 10). A continuous trajectory in the true grid thus appears to the agent as a sequence of action-conditioned jumps, and the agent must implicitly invert the cell permutation to navigate.

Observations. The agent receives a three-dimensional position observation. Its first two entries are the permuted position cursor, formed by relabelling the integer part of the true position by c_r and appending the unchanged sub-cell fraction. The third entry is a soft-reset flag that fires for one step at the boundary between inner episodes within a trial. Both c_d and c_r are hidden.

Task distribution. Permutations and reward grids are drawn from two deterministic deduplicated pools, with reward entries sampled iid from $\mathcal{N}(0, 3)$ and clipped to $[-10, 10]$. Each pool is partitioned into 10,000 training entries and a disjoint 10,000 held-out test entries. The i-th task is (c_d^i, c_r^i), drawing one entry from each pool in lockstep. In Figure 1, tasks are sampled from the cross product of the two pools, so that policies can be trained across many reward functions in a fixed permutation or vice versa.

Trial structure and metric. A meta-trial bundles k inner episodes that share the same (c_r, c_d). Inner episodes have a maximum length of 60 steps for $n = 5$ and 80 steps for $n = 9$. Between inner episodes the agent's position is re-randomized, with the soft-reset flag firing for one step. The trial truncates after the k-th episode. We report oracle-normalized return, defined as per-episode return divided by that of an oracle which teleports from the random start cell to $\arg\max c_r$ and remains there. A score of 1.0 matches the oracle.

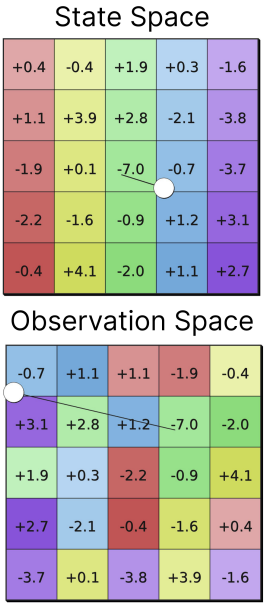

Figure 10: An example ColorGrid transition for $n = 5$.

B.1.2 Wind

Wind is a 2D point meta-RL navigation task introduced by Ni et al. (Ni et al., 2022) as part of their POMDP suite. The agent commands position deltas on the plane, and the world adds a hidden, task-specific wind vector to its motion at every step. A task is an unobserved (w, g) pair in which w perturbs the dynamics while g defines the reward landscape. The two hidden variables stress meta-RL along complementary axes. The wind is identifiable only from the difference between commanded actions and observed motion, and the goal only from reward, so a competent policy must jointly infer the dynamics and search for an unmarked target.

Dynamics and reward. The agent's state is a 2D position $s_t \in \mathbb{R}^2$, advanced by an action $a_t \in [-0.1, 0.1]^2$ and a per-task wind $w \in [-0.08, 0.08]^2$ via $s_{t+1} = s_t + a_t + w$. Inner episodes last 75 steps and terminate early on entry into a small disk around the goal g. The reward is $r_t = e^{-\|s_t - g\|} + \mathbf{1}[s_t \in \text{goal disk}]$, bounded everywhere on the plane with a sharp $+1$ bonus inside the disk.

Observations. The agent receives its own 2D position together with an episode-boundary flag that fires for one step at the boundary between inner episodes within a trial. Both w and g are hidden.

Task distribution. During training, we sample 80 winds uniformly from $[-0.08, 0.08]^2$ and 80 goal locations on the unit circle, with the circle partitioned into the four standard quadrants. One quadrant (Q_2 in our experiments) is held out from the training goal set, so the agent is never rewarded for entering the upper-left region of the plane during training. The two test environments measure distinct generalization gaps that we report separately.

Trial structure and metric. A meta-trial bundles k inner episodes that share the same (w, g). The agent's position resets to the origin between inner episodes, with the soft-reset flag firing for one step, but the task persists across the trial so the policy can use early episodes to identify (w, g) and the rest to exploit them. We train with $k = 3$ and evaluate at $k = 35$ to probe how the policy continues adapting beyond its training horizon. We report mean episodic return.

B.2 Metaworld

We base our experiments on the MetaWorld manipulation suite (Yu et al., 2020), with the reward and goal-masking corrections of McLean et al. (2025), run through the GPU-accelerated MTBench reimplementation (Joshi et al., 2025) for batched simulation. Tasks are simulated in NVIDIA IsaacGym (Makoviychuk et al., 2021) with a Franka Panda arm, and episodes are 250 steps.

Tasks and goals. MetaWorld consists of 50 single-arm tabletop manipulation task families (sliding, picking, button-pressing, door opening, etc.), each parameterized by a goal vector specifying a target site or object position drawn from a task-specific range. Each training task family exposes 100 sampled goal variations, giving ML45 a total of 4,500 train configurations across 45 families and 5 held-out test families. The goal vector is masked to zero in the agent's observation at every timestep, so the agent must infer both the task family and the specific goal from interaction alone.

Action and observation spaces. The action is a 4-D continuous vector $a \in [-1, 1]^4$. Three components specify a Cartesian end-effector delta and the fourth controls the parallel gripper. The observation is a 39-D continuous vector containing the end-effector position and gripper, the position and quaternion of up to two task-relevant objects (zero-padded for tasks with fewer), the previous-timestep object pose, and the goal slot (masked as described above).

Reward. At each timestep, the agent receives the sum of a dense shaping term and a sparse success bonus. MetaWorld reward functions have changed several times: MetaWorld v2 placed rewards on a more consistent scale across tasks to avoid indirectly leaking task identity, while MTBench reduced the dense reward magnitude by .01× to reduce returns and improve multi-task RL. Our experiments avoid this issue with categorical value function networks, so we restore the original dense reward scale, as dense rewards are expected to improve OptiBFM online task inference.

B.3 Humanoid Locomotion

Following Li et al. (2025), we create environments in IsaacLab (Mittal et al., 2025) for humanoids with locomotion tasks. Basically, the humanoids are evaluated to do motion tracking on diverse and challenging motion datasets, e.g. LAFAN (Harvey et al., 2020).

Although RL methods on robotics usually include domain randomization to achieve robust sim-to-real transfer, we intentionally try to create more dynamic and random environments, for example, different wind and gravity. So apart from the domain randomization on mass, friction and so on, we randomly sample the scale and direction of wind, and gravity scale as shown in the figure below. Dynamics setting details are shown in Table. 12.

B.3.1 Representations Details

State and Observation. The privileged state $s_t \in \mathbb{R}^{463}$ includes root height, full body pose, body orientation, as well as linear and angular velocities. The observable state is defined as $o_t \triangleq \left(q_t - \bar{q}, \dot{q}_t, \frac{1}{4}\omega_t^{\text{root}}, g_t\right) \in \mathbb{R}^{64}$, where $q_t \in \mathbb{R}^{29}$ is the joint position normalized by the nominal pose $\bar{q}$, $\dot{q}_t \in \mathbb{R}^{29}$ is the joint velocity, $\omega_t^{\text{root}} \in \mathbb{R}^3$ is the root angular velocity, and $g_t \in \mathbb{R}^3$ is the projected gravity. To mitigate partial observability, we use an observable history $o_{t,H} \triangleq (o_{t-H}, a_{t-H}, \ldots, o_t) \in \mathbb{R}^{93H+64}$, which concatenates past observations and actions over a horizon of length H. All observation components (except root height in the privileged state) are normalized with respect to the humanoid's current facing direction and root position. During BFMs training, we assume access to a dataset of

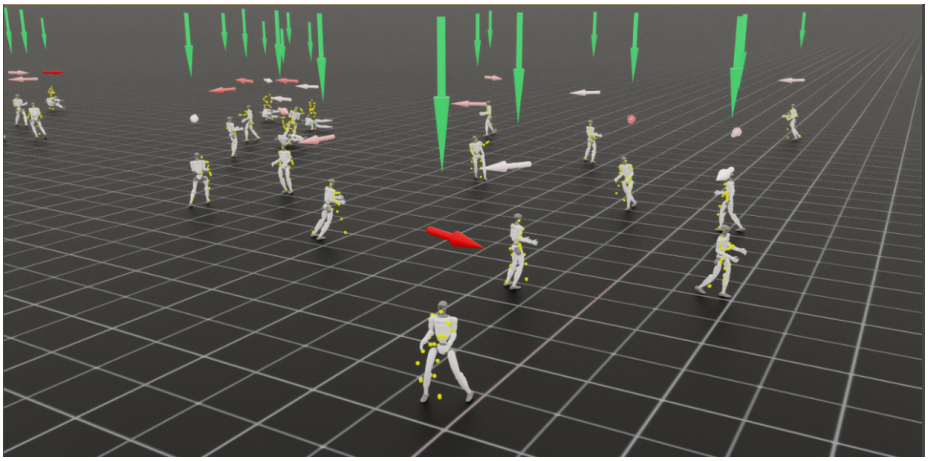

Figure 11: **Wind/Gravity illustration.** During online data collection, the wind and gravity strength in each environment are different – the horizontal arrows indicate the direction and strength of the wind (red as the strongest, white as the weakest), while the length of vertical arrows indicate the strength of gravity.

Domain Randomization

Parameter	**Range**
COM Offset [m]	$\mathcal{U}([-0.02, 0.02])$
Link Mass	$\mathcal{U}([0.95, 1.05])$
Friction	$\mathcal{U}([-0.5, 1.25])$
Default Joint Pos [m]	$\mathcal{U}([-0.02, 0.02])$
Push Robots [m/s]	$\mathcal{U}([0, 0.5])$
Wind Direction[Rad]	$\mathcal{U}([-\pi, \pi])$
Wind Magnitude	$\mathcal{U}([0.0, 30.0])$
Gravity Magnitude[**g**]	$\mathcal{U}([0.8, 1.2])$

Additive Observation Noise

Observation	**Range**
$q_t - \bar{q}$	$\mathcal{U}([-0.01, 0.01])$
$\dot{q}_t$	$\mathcal{U}([-0.5, 0.5])$
grav_t	$\mathcal{U}([-0.05, 0.05])$
$\dot{\omega}_t^{\text{root}}/4$	$\mathcal{U}([-0.05, 0.05])$

Figure 12: Details in training environment.

unlabeled motion trajectories $\mathcal{D} = \{\tau\}$, where each trajectory $\tau = (o_1, s_1, \ldots, o_T, s_T)$ contains both observable and privileged states.

Action. Following PHC, we use a proportional-derivative (PD) controller at each DoF of the humanoid, with the action a_t specifying the PD target. With the target joint set as $q_t^d = a_t$, the torque applied at each joint is $\tau^i = k^p \circ (a_t - q_t) - k^d \circ \dot{q}_t$. This is different from the residual action representation used in prior motion imitation methods, where the action is added to the reference pose:$q_t^d = \hat{q}_t + a_t$ to speed up training. We do not use any external forces or meta-PD control. For the 29-degree-of-freedom (DoF) humanoid we use – Unitree G1, the action $a \in R^{29}$ contains the PD controller targets for all DoFs.

B.3.2 Illustrations of BFM's Failure on Humanoid Locomotion Tasks

BFMs trained with multi-dynamics data might suffer from taking actions from the wrong environment dynamics. This can be shown directly via the visualizations in Fig. 13. For BFM-Multi, in the stage of trying to get up from the ground, the robot acts as there exists wind from certain directions, loses balance and finally tracks badly. As a comparison, BFMs with context learner can identify the dynamics, so the states are close to the BFM-Single, which is trained solely on one type of dynamics.

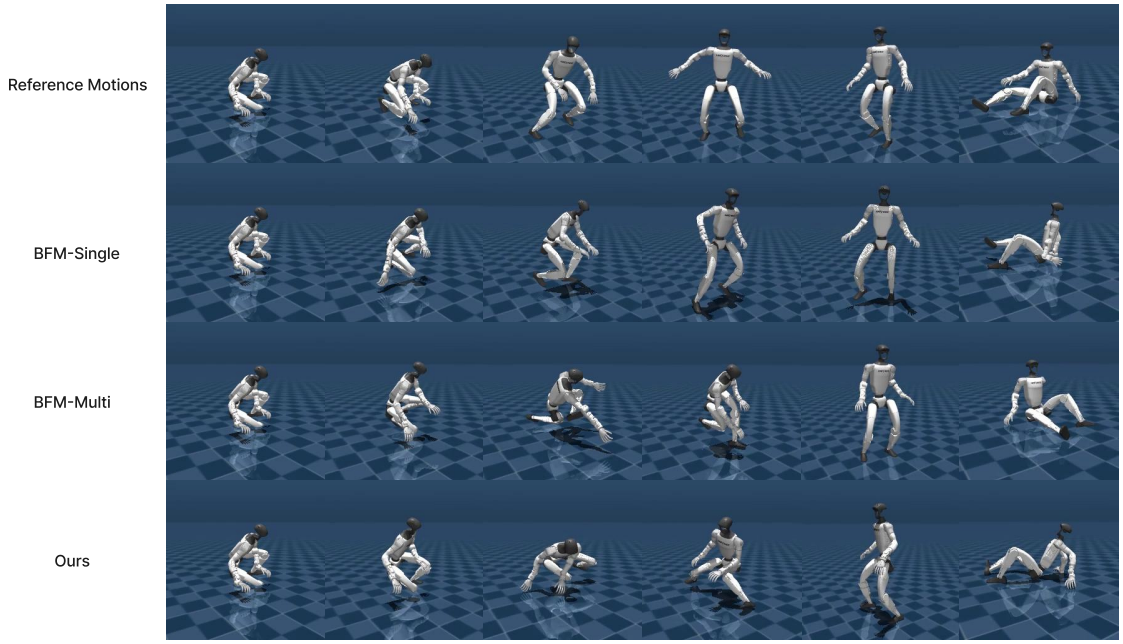

Figure 13: **Qualitative Comparison on Humanoid Motion Tracking.**

Algorithm 1 Latent Context Learning

Input: trajectory batch $\tau = \{\tau^{(i)}\}_{i=1}^{B}$ with $\tau^{(i)} = \{(s_0, a_0, \ldots, s_T)\}$; EMA rate ρ; networks $E_\theta, S_\theta, \mathrm{LDM}_\theta, \mathrm{IDM}_\theta, \mathrm{FDM}_\theta$; EMA target encoder E^-.
Output: augmented trajectory $\bar{\tau} = \{(\bar{s}_t, a_t, \bar{s}_{t+1})\}_t$ with $\bar{s}_t = (s_t, h_t, \tilde{c}_t)$.

// Augment each τ with dynamics features
$h_t = E_\theta(s_t, a_{t-2}),\ \ h_t^- = E^-(s_t, a_{t-2}),\ \ \tilde{c}_t = S_\theta(\tau_{:t}),\ \ \bar{s}_t = (s_t, h_t, \tilde{c}_t).$
// Self-supervised dynamics losses
$\widehat{\mathcal{L}}_{\mathrm{LDM}} = -\frac{1}{N}\sum_{i,t} \cos\big(\mathrm{LDM}_\theta(h_t, a_t, \tilde{c}_t),\ h_{t+1}^-\big)$
$\widehat{\mathcal{L}}_{\mathrm{IDM}} = \frac{1}{2N}\sum_{i,t} \big\|\mathrm{IDM}_\theta(h_t, h_{t+1}, \tilde{c}_t, \tilde{c}_{t+1}) - a_t\big\|^2$
$\widehat{\mathcal{L}}_{\mathrm{FDM}} = \frac{1}{2N}\sum_{i,t} \big\|\mathrm{FDM}_\theta(s_t, a_t, \mathrm{sg}(\tilde{c}_t)) - s_{t+1}\big\|^2$ *// $\tilde{c}_t$ detached by default (diagnostic only)*
$\widehat{\mathcal{L}}_{\mathrm{REG}} = -\frac{1}{B(B-1)\,T}\sum_{i\neq j,\,t}\big[\,1 - \cos\big(\tilde{c}_t^{(i)},\ \tilde{c}_t^{(j)}\big)\,\big]$
Take a gradient step on θ to minimize a weighted sum of (normalized) $\widehat{\mathcal{L}}_{\mathrm{LDM}}, \widehat{\mathcal{L}}_{\mathrm{IDM}}, \widehat{\mathcal{L}}_{\mathrm{FDM}}, \widehat{\mathcal{L}}_{\mathrm{REG}}$.
EMA target update: $E^- \leftarrow (1-\rho)\,E^- + \rho\,E_\theta$.
return augmented trajectory $\bar{\tau}$.

C Implementation Details

C.1 Latent Context Learning

Algorithm 1 summarizes the Latent Context Learning (LCL) procedure, which augments states with features used for BFM training and intrinsic rewards. Per-domain choices are summarized in Table 2. Notably, MetaWorld uses a feed-forward dynamics encoder, while ColorGrid and Wind use small Transformer encoders. The optional cross-trajectory regularizer is disabled in Section 4, though preliminary experiments in Figure 4 enabled it.

Table 2: Latent Context Learning hyperparameters.

Hyperparameter	**ColorGrid**	**Wind**	**MetaWorld**
Latent feature dim ($\dim h$)	32	4	12
Inverse-dynamics feature dim	32	4	12
Inverse-dynamics MLP hidden width	400	128	300
Latent-dynamics predictor hidden width	512	128	300
Diagnostic probe hidden width	400	128	256
S_θ (d_{model}, layers, heads)	Tformer (512, 6, 8)	Tformer (128, 2, 4)	FF (128, 2, –)

Shared across domains: latent-dynamics and inverse-dynamics loss weights both 1; EMA target rate ρ=5×10^{-3}; sphere-normalized features; cross-trajectory regularizer weight 0; probe gradients detached.

Table 3: BFM (TD-JEPA) hyperparameters.

Hyperparameter	**ColorGrid**	**Wind**	**MetaWorld**
ϕ-encoder output dim ($\dim \phi$)	200	160	220
ψ-encoder output dim ($\dim \psi$)	50	40	80
ϕ-predictor hidden width	1024	800	1024
ψ-predictor hidden width	1024	800	1024
Successor-feature discount (γ_{sf})	0.97	0.97	0.98
Reward multiplier (SF target / BLR)	10	10	1
ϕ–ψ orthogonality weight	0.10	0.25	0.10

In reference to notation in Bagatella et al. (2025). Shared across domains: two-tower predictor with three layers (one shared trunk plus two per-head tower layers).

C.2 BFM

While many BFM updates could in principle be used interchangeably, our results use TD-JEPA (Bagatella et al., 2025), primarily because it shares the most surface area with off-policy actor–critic RL. The two heads ϕ (state representation) and ψ (task representation) each use a two-tower predictor, sharing a small input trunk before splitting into per-head towers. We additionally apply a cross-trajectory orthogonality regularizer on the predicted features. A scalar reward multiplier (ten for ColorGrid and Wind, one for MetaWorld) is applied to the per-step reward inside both the SF prediction target and the BLR likelihood at deployment, so it sets the natural magnitude of the regression target $r \approx \langle \psi(s'), z \rangle$ and therefore couples directly to the Thompson-sampling noise level σ discussed in Appendix C.3.

C.3 Online Task Inference

Following OptiBFM (Rupf et al., 2025), we estimate the latent task vector z^* online with Bayesian linear regression on the forward features ϕ, resampling at every timestep. Our experiments use Thompson sampling rather than the optimistic alternative because it scales to the long meta-rollouts in our evaluations, and preliminary experiments found little performance gap. The posterior is governed by an observation-noise scale σ and prior precision λ; we found σ to be the key hyperparameter, scaling roughly linearly with the typical magnitude of the scaled per-step reward.

Table 4: Online task inference (Thompson Sampling) hyperparameters.

Hyperparameter	ColorGrid	Wind	MetaWorld
σ	10	15†	1

† *Shared across domains: Bayesian linear regression with prior precision λ=1, posterior decay 1, exploration bias κ=0, posterior resampled every environment step.*

Table 5: Intrinsic reward hyperparameters.

Hyperparameter	ColorGrid	Wind	MetaWorld
Latent-dynamics-error bonus weight	0.0	0.0	0.0
Episodic novelty bonus weight	1.0	1.0	1.0
Intrinsic critic reward scale	1.0	1.0	1.0
Episodic novelty: nearest neighbors k	10	10	10
Episodic novelty: cluster distance threshold	8×10^{-3}	1×10^{-3}	8×10^{-3}
Episodic novelty: maximum kernel similarity	8.0	8.0	8.0
Episodic novelty: kernel ϵ	1×10^{-4}	1×10^{-5}	1×10^{-4}

C.4 Intrinsic Rewards

The intrinsic reward at each timestep is a sum of two terms: an episodic novelty bonus computed in the style of NGU (Badia et al., 2020) on the latent context features, and a term proportional to the previous-timestep latent-dynamics prediction error (returned by Alg. 1). The combined scalar is normalized and exposed to the policy as an additional observation, so the actor can directly condition on the current intrinsic signal in addition to driving the β-blended critic.

C.5 Actor–Critic Update

Sequence backbone. The actor and the intrinsic–extrinsic critic share a transformer trajectory encoder with rotary position embeddings (Su et al., 2024) and sliding-window FlashAttention (Dao et al., 2022).

Actor and Critic. The critic, Q_R, is a small ensemble of two-hot regression heads over a fixed return range (Hafner et al., 2023), which helps absorb the large variation in TD-target scale induced by randomizing α and β. Both the actor and critic heads consume a Fourier encoding of the (α, β) scalars in Equation 4 as additional input features. The actor is an MLP with a standard Tanh-Gaussian output.

C.6 Computational Resources and Statistical Significance

All models are trained on an NVIDIA A500 GPU, with training runs for the `MetaBFM` (Section 4) experiments lasting 12–36 hours depending on the domain. Shaded error bars in results figures indicate the mean and standard deviation of at least three trials. Results indicating test-time adaptation curves (performance vs. k) average these results over the last 3 evaluation cycles during training, then over seeds.

Table 6: Actor–critic hyperparameters.

Hyperparameter	**ColorGrid**	**Wind**	**MetaWorld**
Training sequence length	360	226	501
Actor / critic context length	96	128	128
Policy attention window	64 timesteps	full attention	64 timesteps
Actor MLP hidden width	512	300	400
Critic MLP hidden width	300	300	400
Critic output bins (two-hot)	64	64	96
Critic return range $[r_{\min}, r_{\max}]$	$[-400, 600]$	$[-50, 400]$	$[-300, 3000]$
Target network rate (τ)	5×10^{-3}	3×10^{-3}	5×10^{-3}
Number of Fourier frequencies for (α, β) encoding	16	8	8

D Limitations

BFM and ZSRL algorithms are an active area of research, and reported gains often depend on hyperparameters, implementation details, and benchmark choice. Our experiments use TD-JEPA as a representative state-of-the-art BFM update, but broader comparisons across BFM objectives and online task inference methods remain important future work. `MetaBFM` also introduces test-time choices through α and β. These parameters are useful because they let a single policy interpolate between exploration, exploitation, and ZSRL-style reward inference, but they also create a model-selection problem. In our experiments, we evaluate a few interpretable settings; in deployment, one may need validation tasks or an online rule for choosing α and β.

E Additional Results

E.1 Online ZSRL

BFMs are inherently proposed as offline algorithms (Touati et al., 2022; Agarwal et al., 2025b), trained on offline exploratory datasets, which can then be used to directly infer policies for diverse reward functions. While the inference involves solving a linear regression problem, as discussed in A.2, using samples drawn from the same offline datasets. Prior work (Sun et al., 2025) has shown that these BFMs are not effective for online training. Methods such as (Li et al., 2025; Tirinzoni et al.) have used expert datasets to guide the online training of these models.

We compare the performance of BFMs when trained offline and online on standard DM Control tasks (Tassa et al., 2018). We compare (Figure 14) the offline-trained BFM (on ExoRL datasets (Denis Yarats, 2022)) using offline inference (labeled *offline*) with the same model using online inference (labeled *offline-online*) as proposed by (Rupf et al., 2025) and with a fully online BFM that does not assume access to any external dataset during training or inference.

It is clear that BFMs are optimally suited to fully offline settings. While using online inference degrades the performance slightly, the performance of online-trained BFM is significantly worse. BFMs require high coverage and well-explained data, which is hard to obtain during online training, leading to poor performance. On the other hand, Meta RL often improves when trained online, as observed from Figure 15.

E.2 Utilizing Available Rewards

BFMs are designed to be trained using reward-free interactions. We design a training procedure by providing occasional reward information to the BFMs to see whether they can make use of the

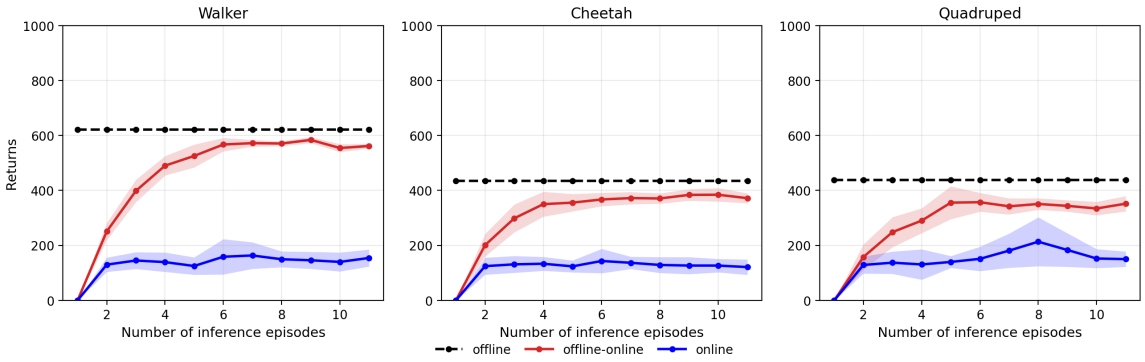

Figure 14: **Offline vs Online BFMs:** Comparison of fully offline, offline training and online inference and fully online BFMs on DM Control tasks.

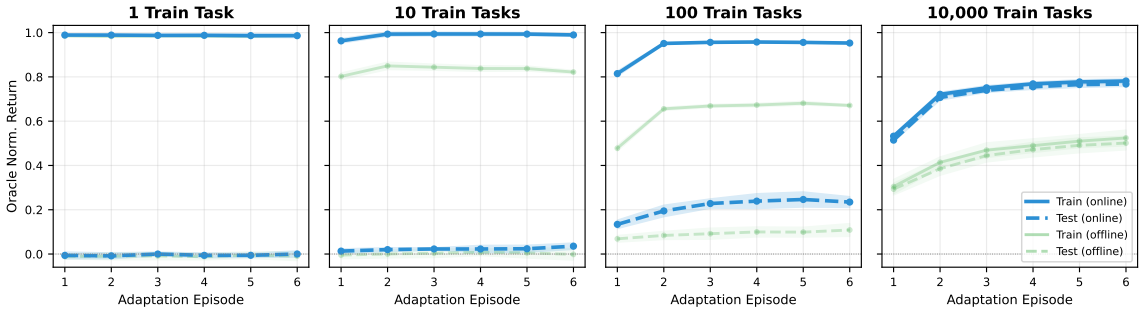

Figure 15: **Online vs. Offline Meta-RL.** In ColorGrid ($n = 5$), offline meta-RL from a static dataset generated by a uniform behavior policy leads to significant distribution shift in long-context representations; online learning substantially improves performance.

available rewards. On the trajectories where rewards are available, the BFMs exploit the reward function to interact with the environment using an optimal policy (optimal for the partially trained BFM). It adds regularization to the state features to ensure they remain sufficiently predictive of the available rewards, as discussed in Section 3.3.

We compare the performance of online-trained BFMs with and without the use of reward functions in Figure 16. The improvement in performance when using available rewards is not consistent. While an improvement is observed in the walker, similar improvements are not observed in the other two environments. On ColorGrid, the BFMs show improvement in performance when leveraging the available rewards, as shown in Figure 17.

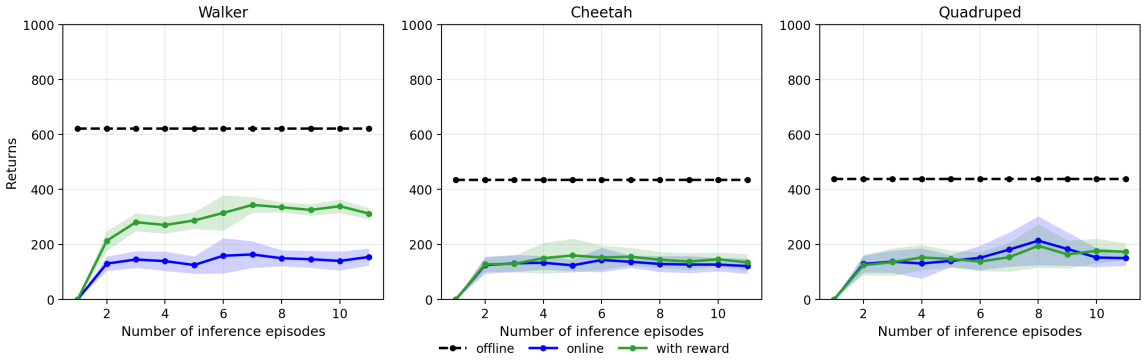

Figure 16: **Effect of using rewards during training of BFMs:** Comparing online BFMs with and without using available rewards during training.

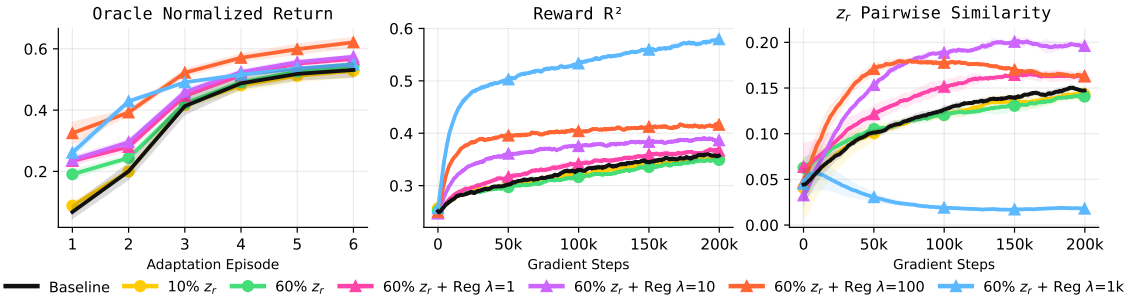

Figure 17: **Reward Regression and z-sampling.** Sampling reward-estimated z values and fitting known reward labels (using an auxiliary loss with weight λ) fits better linear reward estimates (center), focuses learning on more similar latent tasks (right), and marginally improves early-episode adaptation in the ColorGrid domain (left).